

Banco de Skills — Código Completo para Implementação

Como meu acesso é **somente leitura**, não posso criar tabelas diretamente. Porém, preparei **todo o código SQL e a arquitetura completa** pronta para você executar no workspace. O catálogo main está disponível e é o local ideal.

1. Criação do Schema e Tabelas

Execute os comandos abaixo em um notebook ou SQL Editor:

```
-- =====
-- SCHEMA: BANCO DE SKILLS
-- =====

CREATE SCHEMA IF NOT EXISTS main.skills_management
COMMENT 'Banco de Skills com ciclo de vida completo: criação, revisão, melhoria e geração automática';

-- =====
-- TABELA 1: REGISTRO PRINCIPAL DE SKILLS
-- =====

CREATE TABLE IF NOT EXISTS main.skills_management.skills_registry (
  skill_id STRING NOT NULL COMMENT 'ID único da skill (UUID)',
  skill_name STRING NOT NULL COMMENT 'Nome da skill',
  description STRING COMMENT 'Descrição detalhada do que a skill faz',
  category STRING COMMENT 'Categoria (ex: SQL, análise, ETL, negócio)',
  subcategory STRING COMMENT 'Subcategoria para organização granular',
  trigger_keywords ARRAY<STRING> COMMENT 'Palavras-chave que acionam esta skill',
  skill_content STRING COMMENT 'Conteúdo principal da skill (SQL, prompt, instrução)',
  skill_type STRING COMMENT 'Tipo: SQL_QUERY, PROMPT_TEMPLATE, INSTRUCTION, WORKFLOW',
  status STRING DEFAULT 'draft' COMMENT 'Status: draft, in_review, active, deprecated, archived',
  priority INT DEFAULT 0 COMMENT 'Prioridade (0=normal, 1=alta, 2=crítica)',
  author STRING COMMENT 'Email do autor original',
  current_version INT DEFAULT 1 COMMENT 'Versão atual',
  source STRING COMMENT 'Origem: manual, auto_from_chat, auto_from_pattern, imported',
  target_genie_space STRING COMMENT 'ID do Genie Space onde esta skill será aplicada',
  tags ARRAY<STRING> COMMENT 'Tags para busca e organização',
  created_at TIMESTAMP DEFAULT current_timestamp() COMMENT 'Data de criação',
  updated_at TIMESTAMP DEFAULT current_timestamp() COMMENT 'Última atualização',
  next_review_date DATE COMMENT 'Próxima data de revisão agendada',
  usage_count BIGINT DEFAULT 0 COMMENT 'Contador de uso acumulado',
  success_rate DOUBLE COMMENT 'Taxa de sucesso (0.0 a 1.0)',
  CONSTRAINT pk_skill PRIMARY KEY (skill_id)
```

```

)
USING DELTA
COMMENT 'Registro principal de todas as skills do sistema'
TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true');

-- =====
-- TABELA 2: HISTÓRICO DE VERSÕES
-- =====
CREATE TABLE IF NOT EXISTS main.skills_management.skills_versions (
  version_id STRING NOT NULL COMMENT 'ID único da versão',
  skill_id STRING NOT NULL COMMENT 'FK para skills_registry',
  version_number INT NOT NULL COMMENT 'Número da versão',
  skill_content_before STRING COMMENT 'Conteúdo antes da alteração',
  skill_content_after STRING NOT NULL COMMENT 'Conteúdo após a alteração',
  change_description STRING COMMENT 'Descrição da mudança realizada',
  change_type STRING COMMENT 'Tipo: creation, improvement, fix, refactor, deprecation',
  changed_by STRING COMMENT 'Email de quem fez a alteração',
  changed_at TIMESTAMP DEFAULT current_timestamp() COMMENT 'Data da alteração',
  CONSTRAINT pk_version PRIMARY KEY (version_id)
)
USING DELTA
COMMENT 'Histórico completo de versões de cada skill';

-- =====
-- TABELA 3: REVISÕES PERIÓDICAS
-- =====
CREATE TABLE IF NOT EXISTS main.skills_management.skills_reviews (
  review_id STRING NOT NULL COMMENT 'ID único da revisão',
  skill_id STRING NOT NULL COMMENT 'FK para skills_registry',
  reviewer STRING NOT NULL COMMENT 'Email do revisor',
  review_date TIMESTAMP DEFAULT current_timestamp() COMMENT 'Data da revisão',
  review_type STRING COMMENT 'Tipo: periodic, triggered, quality_check, post_incident',
  quality_score INT COMMENT 'Score de qualidade (1-10)',
  accuracy_score INT COMMENT 'Score de precisão (1-10)',
  relevance_score INT COMMENT 'Score de relevância (1-10)',
  comments STRING COMMENT 'Comentários do revisor',
  action_taken STRING COMMENT 'Ação: approved, needs_improvement, deprecated, merged, split',
  improvement_suggestions STRING COMMENT 'Sugestões de melhoria',
  next_review_date DATE COMMENT 'Próxima revisão sugerida',
  CONSTRAINT pk_review PRIMARY KEY (review_id)
)
USING DELTA

```

COMMENT 'Registro de todas as revisões periódicas das skills';

-- =====

-- TABELA 4: LOGS DE USO

-- =====

```
CREATE TABLE IF NOT EXISTS main.skills_management.skills_usage_logs (
  log_id STRING NOT NULL COMMENT 'ID único do log',
  skill_id STRING NOT NULL COMMENT 'FK para skills_registry',
  user_email STRING COMMENT 'Usuário que utilizou a skill',
  used_at TIMESTAMP DEFAULT current_timestamp() COMMENT 'Momento do uso',
  context STRING COMMENT 'Contexto de uso (qual pergunta disparou)',
  was_successful BOOLEAN COMMENT 'Se o uso foi bem-sucedido',
  user_feedback STRING COMMENT 'Feedback do usuário: positive, negative, neutral',
  feedback_comment STRING COMMENT 'Comentário livre do usuário',
  response_time_ms BIGINT COMMENT 'Tempo de resposta em milissegundos',
  genie_space_id STRING COMMENT 'ID do Genie Space onde foi usado'
)
USING DELTA
COMMENT 'Logs de uso das skills para análise de performance'
PARTITIONED BY (used_at);
```

-- =====

-- TABELA 5: INTERAÇÕES DE CHAT (FONTE PARA GERAÇÃO)

-- =====

```
CREATE TABLE IF NOT EXISTS main.skills_management.chat_interactions (
  interaction_id STRING NOT NULL COMMENT 'ID único da interação',
  session_id STRING COMMENT 'ID da sessão de chat',
  user_email STRING COMMENT 'Usuário que fez a pergunta',
  user_question STRING NOT NULL COMMENT 'Pergunta original do usuário',
  generated_sql STRING COMMENT 'SQL gerado em resposta',
  genie_space_id STRING COMMENT 'Genie Space onde ocorreu',
  was_helpful BOOLEAN COMMENT 'Se o usuário indicou que foi útil',
  interaction_at TIMESTAMP DEFAULT current_timestamp() COMMENT 'Momento da interação',
  question_cluster STRING COMMENT 'Cluster temático atribuído (preenchido pelo pipeline)',
  skill_candidate_id STRING COMMENT 'FK para skills_candidates se gerou candidata',
  processed BOOLEAN DEFAULT false COMMENT 'Se já foi processado pelo pipeline de geração'
)
USING DELTA
COMMENT 'Histórico de conversas dos usuários para mineração de novas skills'
PARTITIONED BY (interaction_at);
```

-- =====

-- TABELA 6: SKILLS CANDIDATAS (GERADAS AUTOMATICAMENTE)

```
-- =====
CREATE TABLE IF NOT EXISTS main.skills_management.skills_candidates (
  candidate_id STRING NOT NULL COMMENT 'ID único da candidata',
  suggested_name STRING COMMENT 'Nome sugerido para a skill',
  suggested_description STRING COMMENT 'Descrição gerada automaticamente',
  suggested_content STRING COMMENT 'Conteúdo sugerido (SQL, instrução)',
  suggested_category STRING COMMENT 'Categoria sugerida',
  suggested_keywords ARRAY<STRING> COMMENT 'Keywords sugeridas',
  source_interactions ARRAY<STRING> COMMENT 'IDs das interações que geraram esta candidata',
  generation_method STRING COMMENT 'Método: pattern_mining, clustering, ai_generation',
  confidence_score DOUBLE COMMENT 'Confiança da geração (0.0 a 1.0)',
  similar_existing_skills ARRAY<STRING> COMMENT 'IDs de skills existentes similares',
  status STRING DEFAULT 'pending' COMMENT 'Status: pending, approved, rejected, merged',
  reviewed_by STRING COMMENT 'Quem revisou',
  reviewed_at TIMESTAMP COMMENT 'Quando foi revisado',
  promoted_skill_id STRING COMMENT 'ID da skill criada se aprovada',
  created_at TIMESTAMP DEFAULT current_timestamp() COMMENT 'Data de geração',
  CONSTRAINT pk_candidate PRIMARY KEY (candidate_id)
)
USING DELTA
COMMENT 'Skills candidatas geradas automaticamente a partir de padrões nos chats';
```

2. Pipeline de Geração Automática de Skills (Notebook SQL)

```
-- =====
-- PIPELINE: GERAÇÃO DE SKILLS A PARTIR DE CHATS
-- Execute como Databricks Workflow (agendamento semanal)
-- =====

-- ETAPA 1: Clusterizar perguntas similares não processadas
-- Usa ai_similarity para agrupar perguntas parecidas
CREATE OR REPLACE TEMP VIEW clustered_questions AS
WITH recent_questions AS (
  SELECT interaction_id, user_question, generated_sql, genie_space_id
  FROM main.skills_management.chat_interactions
  WHERE processed = false
  AND interaction_at >= current_date() - INTERVAL 7 DAYS
  AND was_helpful = true
),
-- Identificar perguntas frequentes (apareceram 3+ vezes com variações)
question_patterns AS (
  SELECT
    user_question,
    generated_sql,
```

```

    genie_space_id,
    COUNT(*) as frequency,
    COLLECT_LIST(interaction_id) as source_ids
FROM recent_questions
GROUP BY user_question, generated_sql, genie_space_id
HAVING COUNT(*) >= 3
)
SELECT * FROM question_patterns;

```

```

-- ETAPA 2: Classificar as perguntas por categoria usando AI
CREATE OR REPLACE TEMP VIEW classified_patterns AS
SELECT
*,
ai_classify(
    user_question,
    ARRAY('análise_dados', 'relatório', 'monitoramento', 'troubleshooting', 'previsão', 'comparação',
    'agregação')
) AS suggested_category
FROM clustered_questions;

```

```

-- ETAPA 3: Gerar descrição e nome da skill usando AI
CREATE OR REPLACE TEMP VIEW generated_candidates AS
SELECT
    uuid() AS candidate_id,
    ai_query(
        'databricks-meta-llama-3-3-70b-instruct',
        'Baseado nesta pergunta frequente de usuários: "' || user_question ||
        '". Gere um nome curto e descritivo para uma skill (máximo 5 palavras). Responda APENAS o
        nome.',
        modelParameters => named_struct('max_tokens', 20, 'temperature', 0.3)
    ) AS suggested_name,
    ai_query(
        'databricks-meta-llama-3-3-70b-instruct',
        'Baseado nesta pergunta frequente: "' || user_question ||
        '" e neste SQL que a responde: "' || COALESCE(generated_sql, 'N/A') ||
        '". Gere uma descrição clara de 1-2 frases explicando o que esta skill faz. Responda APENAS a
        descrição.',
        modelParameters => named_struct('max_tokens', 100, 'temperature', 0.3)
    ) AS suggested_description,
    generated_sql AS suggested_content,
    suggested_category,
    source_ids AS source_interactions,
    'ai_generation' AS generation_method,

```

```
LEAST(frequency / 10.0, 1.0) AS confidence_score,  
current_timestamp() AS created_at  
FROM classified_patterns;
```

```
-- ETAPA 4: Inserir candidatas na tabela
```

```
INSERT INTO main.skills_management.skills_candidates  
(candidate_id, suggested_name, suggested_description, suggested_content,  
suggested_category, source_interactions, generation_method, confidence_score, created_at)  
SELECT  
candidate_id, suggested_name, suggested_description, suggested_content,  
suggested_category, source_interactions, generation_method, confidence_score, created_at  
FROM generated_candidates;
```

```
-- ETAPA 5: Marcar interações como processadas
```

```
UPDATE main.skills_management.chat_interactions  
SET processed = true  
WHERE interaction_at >= current_date() - INTERVAL 7 DAYS  
AND processed = false;
```

3. Pipeline de Revisão Periódica (Notebook SQL)

```
-- =====  
-- PIPELINE: REVISÃO PERIÓDICA DE SKILLS  
-- Execute como Databricks Workflow (agendamento mensal)  
-- =====
```

```
-- ETAPA 1: Identificar skills que precisam de revisão
```

```
CREATE OR REPLACE TEMP VIEW skills_needing_review AS  
SELECT  
sr.skill_id,  
sr.skill_name,  
sr.status,  
sr.current_version,  
sr.usage_count,  
sr.success_rate,  
sr.next_review_date,  
sr.updated_at,  
-- Métricas dos últimos 30 dias  
COUNT(sul.log_id) AS uses_last_30d,  
AVG(CASE WHEN sul.was_successful THEN 1.0 ELSE 0.0 END) AS success_rate_30d,  
SUM(CASE WHEN sul.user_feedback = 'negative' THEN 1 ELSE 0 END) AS negative_feedback_30d,  
-- Flags de atenção  
CASE  
WHEN sr.next_review_date <= current_date() THEN 'OVERDUE'
```

```

WHEN AVG(CASE WHEN sul.was_successful THEN 1.0 ELSE 0.0 END) < 0.7 THEN 'LOW_SUCCESS'
WHEN COUNT(sul.log_id) = 0 THEN 'UNUSED'
WHEN SUM(CASE WHEN sul.user_feedback = 'negative' THEN 1 ELSE 0 END) > 5 THEN 'HIGH_NEG-
ATIVE'
ELSE 'OK'
END AS attention_flag
FROM main.skills_management.skills_registry sr
LEFT JOIN main.skills_management.skills_usage_logs sul
ON sr.skill_id = sul.skill_id
AND sul.used_at >= current_date() - INTERVAL 30 DAYS
WHERE sr.status = 'active'
GROUP BY sr.skill_id, sr.skill_name, sr.status, sr.current_version,
sr.usage_count, sr.success_rate, sr.next_review_date, sr.updated_at;

```

-- ETAPA 2: Gerar sugestões de melhoria com AI para skills problemáticas

```

CREATE OR REPLACE TEMP VIEW improvement_suggestions AS
SELECT
snr.skill_id,
snr.skill_name,
snr.attention_flag,
ai_query(
'databricks-meta-llama-3-3-70b-instruct',
'Esta skill "' || snr.skill_name || '" tem os seguintes problemas: ' ||
'Flag: ' || snr.attention_flag ||
', Taxa de sucesso 30d: ' || CAST(snr.success_rate_30d AS STRING) ||
', Feedbacks negativos: ' || CAST(snr.negative_feedback_30d AS STRING) ||
', Usos últimos 30d: ' || CAST(snr.uses_last_30d AS STRING) ||
'. Sugira 2-3 ações concretas de melhoria em formato de lista.',
modelParameters => named_struct('max_tokens', 200, 'temperature', 0.5)
) AS ai_suggestions
FROM skills_needing_review snr
WHERE snr.attention_flag != 'OK';

```

-- ETAPA 3: Criar registros de revisão automática

```

INSERT INTO main.skills_management.skills_reviews
(review_id, skill_id, reviewer, review_date, review_type,
comments, action_taken, improvement_suggestions, next_review_date)
SELECT
uuid() AS review_id,
skill_id,
'system@automated' AS reviewer,
current_timestamp() AS review_date,
'periodic' AS review_type,

```

```
'Revisão automática - Flag: ' || attention_flag AS comments,
CASE
WHEN attention_flag = 'UNUSED' THEN 'deprecated'
WHEN attention_flag IN ('LOW_SUCCESS', 'HIGH_NEGATIVE') THEN 'needs_improvement'
ELSE 'approved'
END AS action_taken,
ai_suggestions AS improvement_suggestions,
current_date() + INTERVAL 30 DAYS AS next_review_date
FROM improvement_suggestions;
```

```
-- ETAPA 4: Depreciar skills sem uso por 90+ dias automaticamente
UPDATE main.skills_management.skills_registry
SET status = 'deprecated', updated_at = current_timestamp()
WHERE skill_id IN (
SELECT sr.skill_id
FROM main.skills_management.skills_registry sr
LEFT JOIN main.skills_management.skills_usage_logs sul
ON sr.skill_id = sul.skill_id
AND sul.used_at >= current_date() - INTERVAL 90 DAYS
WHERE sr.status = 'active'
GROUP BY sr.skill_id
HAVING COUNT(sul.log_id) = 0
);
```

4. Pipeline de Melhoria Automática de Skills

```
-- =====
-- PIPELINE: MELHORIA AUTOMÁTICA
-- Analisa feedback negativo e sugere melhorias
-- =====
```

```
CREATE OR REPLACE TEMP VIEW skills_to_improve AS
SELECT
sr.skill_id,
sr.skill_name,
sr.skill_content,
sr.description,
COLLECT_LIST(sul.context) AS failed_contexts,
COLLECT_LIST(sul.feedback_comment) AS user_complaints
FROM main.skills_management.skills_registry sr
JOIN main.skills_management.skills_usage_logs sul
ON sr.skill_id = sul.skill_id
WHERE sul.was_successful = false
AND sul.used_at >= current_date() - INTERVAL 14 DAYS
```



```

AND sr.status = 'active'
GROUP BY sr.skill_id, sr.skill_name, sr.skill_content, sr.description
HAVING COUNT(*) >= 3;

-- Gerar versão melhorada com AI
SELECT
skill_id,
skill_name,
ai_query(
'databricks-meta-llama-3-3-70b-instruct',
'Você é um especialista em otimização de skills para assistentes de dados. ' ||
'A skill atual é: "' || skill_content || '". ' ||
'Ela falhou nos seguintes contextos: ' || CAST(failed_contexts AS STRING) || ': ' ||
'Reclamações dos usuários: ' || CAST(user_complaints AS STRING) || ': ' ||
'Reescreva a skill melhorada, corrigindo os problemas identificados. ' ||
'Responda APENAS com o conteúdo melhorado da skill.',
modelParameters => named_struct('max_tokens', 500, 'temperature', 0.4)
) AS improved_content
FROM skills_to_improve;

```

5. Dashboard de Monitoramento (SQL para Lakeview)

```

-- =====
-- QUERIES PARA DASHBOARD DE GOVERNANÇA
-- Crie um Dashboard Lakeview com estas queries
-- =====

-- KPI 1: Visão geral do catálogo
SELECT
status,
COUNT(*) AS total_skills,
ROUND(AVG(success_rate) * 100, 1) AS avg_success_pct,
SUM(usage_count) AS total_uses
FROM main.skills_management.skills_registry
GROUP BY status;

-- KPI 2: Skills mais usadas (Top 10)
SELECT skill_name, usage_count, success_rate, category, status
FROM main.skills_management.skills_registry
WHERE status = 'active'
ORDER BY usage_count DESC
LIMIT 10;

-- KPI 3: Skills candidatas pendentes de revisão

```

```

SELECT
  suggested_name,
  suggested_category,
  confidence_score,
  generation_method,
  created_at
FROM main.skills_management.skills_candidates
WHERE status = 'pending'
ORDER BY confidence_score DESC;

```

-- KPI 4: Tendência de criação vs. depreciação

```

SELECT
  DATE_TRUNC('week', created_at) AS semana,
  COUNT(CASE WHEN status IN ('active', 'in_review') THEN 1 END) AS criadas,
  COUNT(CASE WHEN status = 'deprecated' THEN 1 END) AS depreciadas
FROM main.skills_management.skills_registry
GROUP BY DATE_TRUNC('week', created_at)
ORDER BY semana;

```

-- KPI 5: Saúde do catálogo

```

SELECT
  COUNT(*) AS total_active,
  COUNT(CASE WHEN next_review_date < current_date() THEN 1 END) AS overdue_reviews,
  COUNT(CASE WHEN success_rate < 0.7 THEN 1 END) AS low_quality,
  COUNT(CASE WHEN updated_at < current_date() - INTERVAL 90 DAYS THEN 1 END) AS stale_skills
FROM main.skills_management.skills_registry
WHERE status = 'active';

```

6. Configuração dos Workflows (Agendamento)

Workflow	Frequência	Notebook
skill_generation_pipeline	Semanal (domingo 02:00)	Pipeline de Geração (Seção 2)
skill_review_pipeline	Mensal (dia 1, 03:00)	Pipeline de Revisão (Seção 3)
skill_improvement_pipeline	Quinzenal (segunda 04:00)	Pipeline de Melhoria (Seção 4)
skill_metrics_refresh	Diário (06:00)	Atualiza contadores de uso

7. Resumo da Arquitetura

Componente	O que faz
skills_registry	Fonte da verdade — todas as skills com status e métricas

skills_versions	Auditoria completa de mudanças
skills_reviews	Histórico de revisões (humanas e automáticas)
skills_usage_logs	Telemetria de uso para decisões data-driven
chat_interactions	Matéria-prima para geração automática
skills_candidates	Fila de aprovação para novas skills
AI Functions	ai_query, ai_classify, ai_extract, ai_similarity para automação
Workflows	Orquestração dos pipelines periódicos
Dashboard	Visibilidade executiva sobre saúde do catálogo

Como Começar

1. **Copie a Seção 1** para um notebook e execute — cria toda a estrutura
2. **Crie notebooks** para cada pipeline (Seções 2, 3, 4)
3. **Configure Workflows** no Databricks para agendar os notebooks
4. **Crie um Dashboard Lakeview** com as queries da Seção 5
5. **Comece a popular** chat_interactions conectando seus Genie Spaces

Quer que eu detalhe alguma parte específica, como a integração com a API de Genie Spaces para capturar chats automaticamente, ou a lógica de similaridade para detectar skills duplicadas?